

Fast Sentiment Analysis Using Distilled Transformers on CPU

Midterm Report

Arwa Elgazar*, Eman Elsayed*, Esraa Nematalla*

*School of Computing, Queen’s University, Kingston, ON, Canada

Abstract—This midterm report presents results from Weeks 1 and 2 of an investigation into CPU-efficient sentiment analysis using distilled transformers. We train and benchmark `distilbert-base-uncased` against a TF-IDF + Logistic Regression (LR) baseline on two datasets: SST-2 (3,000 stratified short phrases) and IMDb (2,000 stratified full-length reviews). All experiments run exclusively on CPU. DistilBERT achieves 86.0% accuracy on SST-2 (+14.0 pp over the LR baseline) and 89.0% on IMDb (+6.25 pp), with macro-F1 scores of 86.0% and 89.0% respectively. These gains come at a latency cost of $45\times$ on SST-2 (18.5 ms vs. 0.41 ms per sample) and $76\times$ on IMDb (51.4 ms vs. 0.67 ms), and a model-size ratio of $\sim 1,000\times$ (267.9 MB vs. ≤ 2.1 MB). Dataset-specific tokenisation cutoffs (`max_length`=64 for SST-2 and 256 for IMDb) were selected through empirical token-coverage analysis. Remaining work in Weeks 3 and 4 includes INT8 dynamic quantisation, sequence-length and training-size ablations, layer-freezing efficiency curves, and full error analysis.

Index Terms—sentiment analysis, DistilBERT, knowledge distillation, TF-IDF, logistic regression, CPU inference, natural language processing

I. INTRODUCTION

Sentiment analysis, the automatic detection of opinion polarity in text, is one of the most widely deployed natural language processing (NLP) applications, powering product-review aggregation, social-media monitoring, and customer-feedback pipelines. State-of-the-art transformer models achieve near-human accuracy on standard benchmarks, but their computational demands during inference can make deployment prohibitive on resource-constrained hardware: laptops, edge devices, or cloud micro-instances with strict latency budgets.

Knowledge distillation [1] offers a practical middle ground. A compact *student* model is trained to mimic the outputs of a larger *teacher*, retaining much of the teacher’s performance while being substantially faster and lighter. DistilBERT [2] distills BERT [3] into a six-layer transformer with 66.9M parameters (40 % fewer than BERT-base) and roughly twice the inference speed on GPU, while retaining 97 % of BERT’s performance on GLUE.

This project investigates three concrete questions on CPU-only hardware:

- 1) How much accuracy does DistilBERT gain over a strong classical baseline (TF-IDF + Logistic Regression) on binary sentiment tasks with two datasets of contrasting text length?

- 2) What is the quantitative cost in inference latency and model footprint when moving from a classical pipeline to a distilled transformer?
- 3) Can that cost be reduced through post-training INT8 quantisation, and through selective layer-freezing, without meaningful accuracy degradation?

The scope is deliberately narrow: all midterm experiments use stratified subsets (3,000 samples for SST-2, 2,000 for IMDb), run entirely on CPU, and pre-train no model from scratch. The two datasets differ substantially in text length, stressing both models differently and enabling dataset-level efficiency comparisons.

II. RELATED WORK

A. BERT and Pre-trained Language Models

Devlin et al. [3] introduced BERT, establishing that deeply bidirectional pre-training on masked language modelling and next-sentence prediction transfers effectively to a wide range of downstream NLP tasks. Fine-tuning BERT-base achieves 93.5 % accuracy on the full SST-2 training set. The primary limitation for deployment is its 110M-parameter footprint and the quadratic complexity of self-attention with respect to sequence length.

B. DistilBERT: Distillation for Efficient Inference

Sanh et al. [2] proposed DistilBERT as a practical solution to BERT’s inference cost. A triple loss (soft-probability distillation from the teacher, cosine embedding alignment, and masked language modelling on the teacher’s own outputs) allows a six-layer student to retain 97 % of BERT’s GLUE score with 40 % fewer parameters and 60 % faster GPU inference. This work directly motivates our project: we apply DistilBERT in a CPU-only setting and quantify its trade-offs against a lightweight classical method, extending the analysis to include latency, model size, and planned post-training compression.

C. Classical Baselines for Sentiment

TF-IDF-weighted bag-of-words features combined with logistic regression or support-vector machines remain strong baselines [4]. On IMDb, this approach achieves 85 to 90 % accuracy with appropriate bigram features, sub-linear TF scaling, and negation-preserving tokenisation. Its principal advantage is sub-millisecond inference and a serialised size under 10 MB, making the accuracy trade-off context-dependent, which is exactly the trade-off we quantify here.

D. Compression and Efficiency

Post-training INT8 dynamic quantisation [6] converts floating-point linear weights to 8-bit integers at inference time without a calibration dataset. Prior studies report 20 to 40 % latency reductions with < 1 pp accuracy loss on BERT-family models on CPU. Layer-freezing, which trains only the top transformer layers, is a complementary strategy that reduces both training time and, as we hypothesise, CPU inference latency by enabling better JIT optimisation of frozen sub-graphs. These two techniques form the core of Weeks 3 and 4.

III. METHODOLOGY

A. Datasets and Preprocessing

1) *SST-2 (GLUE Benchmark)*: Binary sentiment labels (positive/negative) assigned to short phrase-level fragments from movie reviews, loaded via HuggingFace Datasets ("glue", "sst2") [5]. The full training split contains 67,349 sentences; sentences have a mean length of 9.3 words (median 7.0, max 51). We draw a balanced stratified subset of 3,000 samples (1,500 per class) with a 200-sample held-out validation set.

Token-coverage analysis shows that `max_length=32` covers 98.2 % of sentences, while `max_length=64` achieves 100 % coverage. We therefore set `max_length=64`, accepting zero truncation while keeping sequences short for CPU efficiency.

2) *IMDb*: 25,000 full-length movie reviews for training and 25,000 for testing [4]. Raw reviews contain HTML artefacts; a `clean_imdb()` preprocessing function decodes HTML entities, strips tags, lowercases, and collapses whitespace. The review length distribution is highly right-skewed (median 173 words, p95 592 words, max 2,459 words). A balanced subset of 2,000 samples (1,000 per class) is drawn from the official training split; the official test set (25,000 samples) is held out entirely for the final report.

Token-coverage analysis justifies `max_length=256`: this covers 58.3 % of reviews while running at approximately 2× the speed of `max_length=512` (which covers only 86.5 %). The 80/20 train-to-validation split yields 1,600 training and 400 validation samples.

3) *Preprocessing Pipelines*: Two separate pipelines are maintained throughout, as applying identical cleaning to both models would constitute a methodology error: DistilBERT was pre-trained on raw text, so post-hoc lowercasing or punctuation removal discards casing signals learned during pre-training.

Classical pipeline. Lowercase, collapse whitespace. Punctuation is intentionally retained because negation markers (*not*, *n't*) carry sentiment signal.

DistilBERT pipeline. Raw WordPiece tokenisation via `distilbert-base-uncased` tokenizer, truncated to the dataset-specific `max_length`. Dynamic per-batch padding is applied via `DataCollatorWithPadding` rather than padding all sequences to `max_length` globally, reducing unnecessary computation.

B. Classical Baseline

The baseline pipeline uses scikit-learn's `TfidfVectorizer` (`ngram_range=(1, 2)`, `max_features=50,000`, `sublinear_tf=True`) feeding into a `LogisticRegression` classifier ($C=1.0$, `max_iter=1000`, `lbfgs` solver). Bigrams are used because they capture local negation patterns ("*not good*") that unigrams miss; their benefit over unigrams is confirmed by the ablation in Table III. Stop words are intentionally retained to preserve sentiment-relevant negations. The same pipeline configuration is applied to both datasets without modification.

Inference latency is measured with a 100-sample benchmark preceded by 5 warm-up samples to exclude cold-start effects, reporting both median and 95th-percentile (p95) wall-clock time via `time.perf_counter()`. Model size is estimated as the byte count of the pickled object.

C. DistilBERT Fine-Tuning

We fine-tune `distilbert-base-uncased` (66.9 M parameters) from HuggingFace Transformers [6] by attaching a two-class linear classification head on top of the [CLS] token representation. The full model is trained end-to-end using the HuggingFace Trainer API.

Training hyperparameters follow the BERT fine-tuning recommendations of Devlin et al. [3]:

- Optimizer: AdamW, $\text{lr} = 2 \times 10^{-5}$
- Weight decay: 0.01
- Gradient clipping: 1.0
- Warmup: linear over 10 % of total steps
- Batch size: 16
- Epochs: 2 (both datasets)

The checkpoint with the highest validation accuracy is retained via `load_best_model_at_end=True`. All random seeds (Python `random`, NumPy, PyTorch, `PYTHONHASHSEED=42`) are fixed before any data loading or model initialisation.

IV. PRELIMINARY EXPERIMENTS AND RESULTS

A. Implementation Details

All experiments run on a single workstation with 20 physical / 28 logical CPU cores, 34 GB RAM, and no GPU (CUDA not available). The software stack is: Python 3.14.5, PyTorch 2.12.0+cpu, HuggingFace Transformers 5.8.1, HuggingFace Datasets 4.8.5, scikit-learn 1.8.0. All results are produced with `SEED=42` fixed across all randomness sources.

B. Classical Baseline Results

On SST-2 (Fig. 3), the LR+TF-IDF pipeline achieves 72.0 % accuracy and macro-F1 of 71.9 % in 0.064 s of training time. Per-class breakdown: Negative precision 0.70 / recall 0.77 / F1 0.74; Positive precision 0.74 / recall 0.67 / F1 0.70. Median inference latency is 0.41 ms (p95: 0.55 ms) and the serialised model occupies 0.276 MB.

On IMDb (Fig. 4), the same pipeline achieves 82.75 % accuracy and macro-F1 of 82.74 % in 1.0 s of training. Per-class

SST-2 Dataset EDA | Subset: 3,000 samples | Seed=42

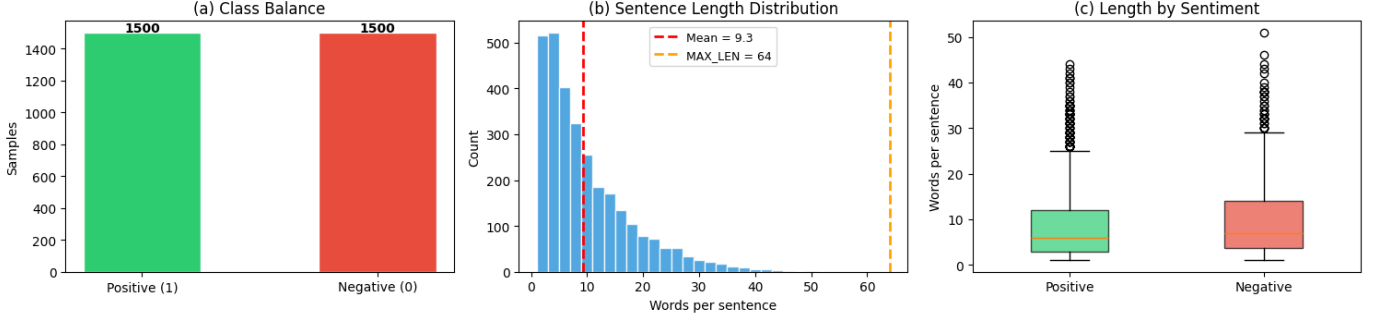


Fig. 1. SST-2 EDA: (a) class balance (3,000 samples, 1,500 per class); (b) word-count histogram (median 7 words, max 51); (c) token-coverage analysis justifying `max_length=64` (100 % coverage).

IMDb Dataset EDA | Subset: 2,000 samples | Seed=42

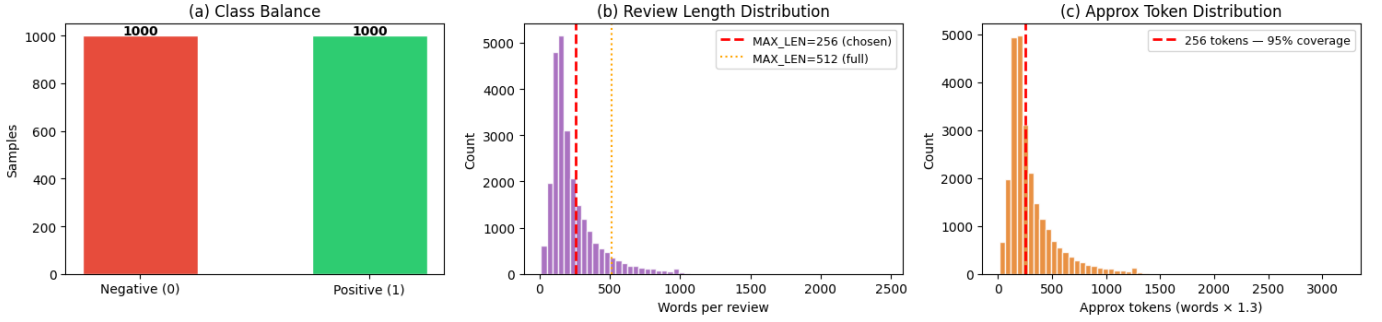


Fig. 2. IMDb EDA: (a) class balance (2,000 samples, 1,000 per class); (b) word-count histogram (median 173, p95 592, max 2,459); (c) approximate token distribution justifying `max_length=256` (58.3 % coverage, 2× faster than 512).

SST-2 Classical Baseline | Acc: 72.00%

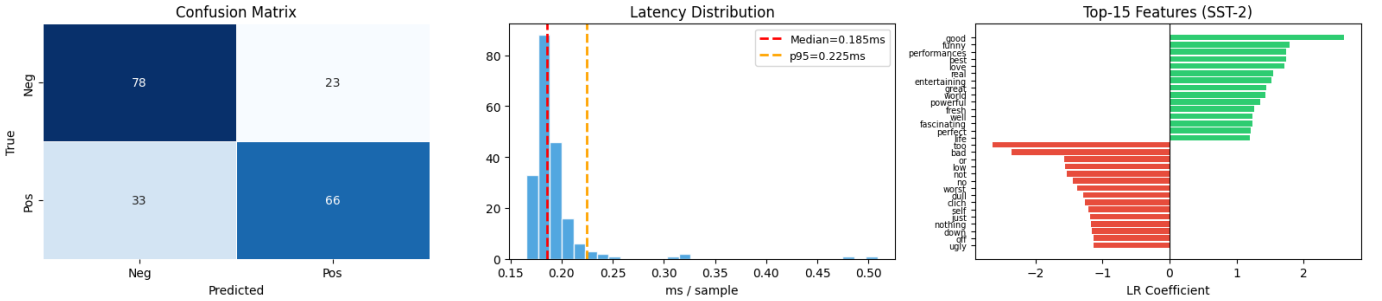


Fig. 3. SST-2 TF-IDF+LR baseline: confusion matrix, per-class precision/recall/F1, and latency distribution (100 samples, 5 warm-up).

breakdown: Negative precision 0.84 / recall 0.81 / F1 0.82; Positive precision 0.81 / recall 0.85 / F1 0.83. Median latency is 0.67 ms (p95: 1.12 ms) and model size is 2.14 MB. The higher IMDb accuracy reflects richer lexical signal available in longer, full-length reviews.

C. DistilBERT Fine-Tuning Results

Table I records the per-epoch training and validation metrics extracted from `trainer.state.log_history`.

On SST-2 (Fig. 5), training loss decreases monotonically from 0.3133 at epoch 1 to 0.1946 at epoch 2. Validation accuracy improves from 82.0 % to 83.0 %, with the best-model

TABLE I
DISTILBERT PER-EPOCH TRAINING METRICS

Dataset	Ep.	Train Loss	Val Loss	Val Acc	Val F1
SST-2	1	0.3133	0.3895	82.0%	0.8197
SST-2	2	0.1946	0.4232	83.0%	0.8299
IMDb	1	0.4049	0.3242	88.0%	0.8800
IMDb	2	0.2681	0.2822	89.0%	0.8900

Best checkpoint selected by highest validation accuracy.

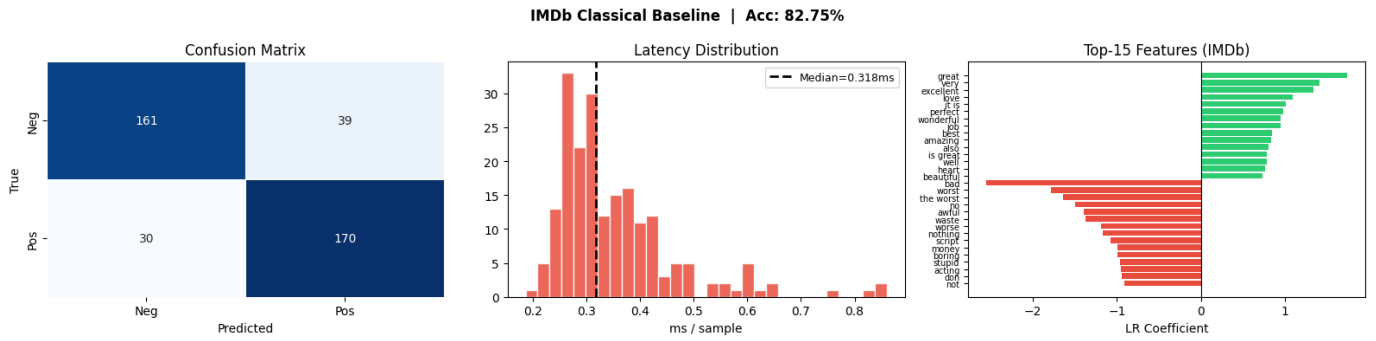


Fig. 4. IMDb TF-IDF + LR baseline: confusion matrix, per-class precision/recall/F1, and latency distribution.

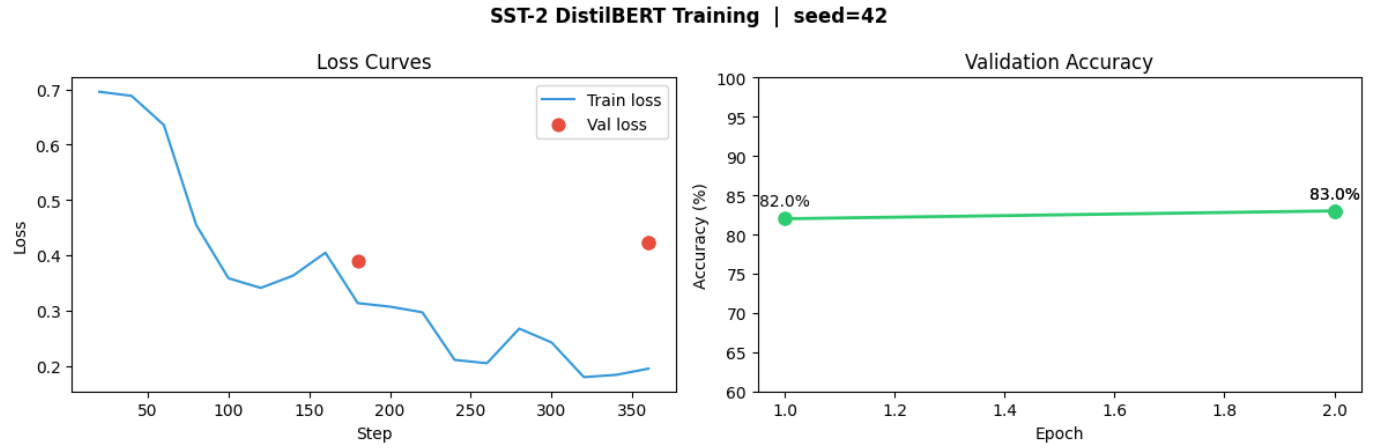


Fig. 5. SST-2 DistilBERT training dynamics (3,000 samples, 2 epochs, seed=42): training-loss curve and validation-accuracy per epoch.

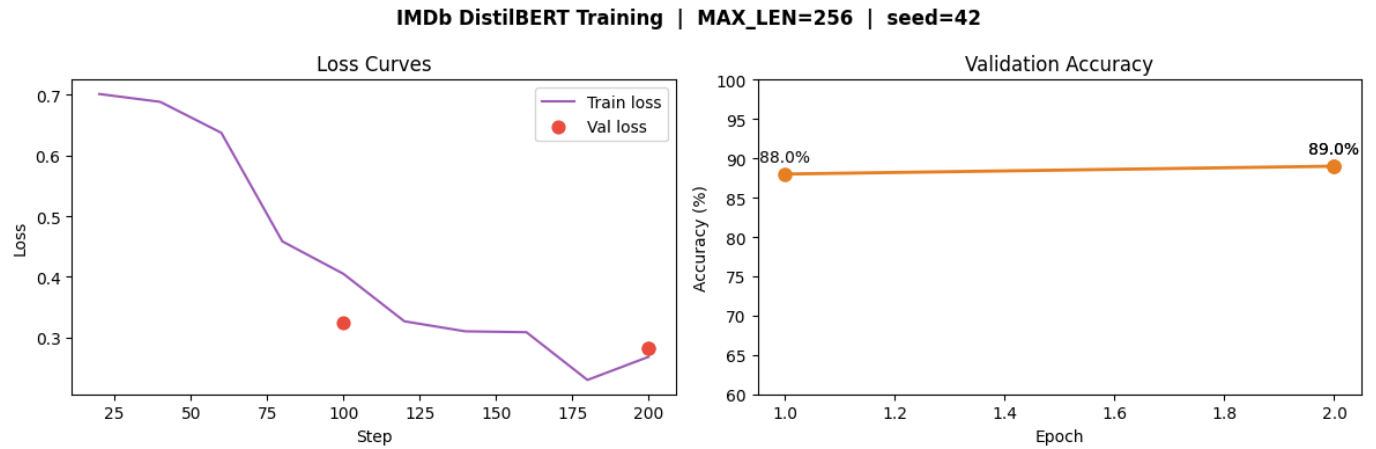


Fig. 6. IMDb DistilBERT training dynamics (1,600 training / 400 validation samples, 2 epochs, seed=42): training-loss curve and validation-accuracy per epoch.

checkpoint retained at epoch 2. Final evaluation accuracy is **86.0 %** and macro-F1 is **85.95 %**, representing a **+14.0 pp** improvement over the LR baseline. Training completed in 7.6 min on CPU.

On IMDb (Fig. 6), training loss decreases from 0.4049 to 0.2681 across two epochs. Validation accuracy improves

from 88.0 % to 89.0 %, indicating steady learning with no sign of overfitting on the 1,600-sample subset. Final evaluation accuracy is **89.0 %** and macro-F1 is **89.0 %**, a **+6.25 pp** gain over the LR baseline. Training completed in 11.5 min on CPU.

The smaller accuracy gap on IMDb (+6.25 pp vs. +14.0 pp for SST-2) is consistent with the expectation that TF-IDF

TABLE II
MIDTERM COMPARISON: ALL MODELS AND DATASETS (VALIDATION SET)

DS	Model	Acc. (%)	F1 (%)	Med (ms)	p95 (ms)	Size (MB)
SST-2	LR + TF-IDF	72.00	71.90	0.41	0.55	0.28
SST-2	DistilBERT	86.00	85.95	18.50	20.20	267.90
IMDb	LR + TF-IDF	82.75	82.74	0.67	1.12	2.14
IMDb	DistilBERT	89.00	89.00	51.40	54.20	267.90

[†]Latency: median/p95, 100 samples, 5 warm-up, FP32 CPU.

[‡]Size: pickle bytes (LR); `torch.save` bytes (DistilBERT).

TABLE III
N-GRAM RANGE ABLATION: TF-IDF BASELINE (SST-2 VALIDATION)

N-gram Range	Val Accuracy (%)	Val F1
(1, 1): unigrams only	70.50	0.7038
(1, 2): unigrams + bigrams	72.00	0.7190
(1, 3): up to trigrams	71.50	0.7142

(1,2) selected; trigrams add noise on short SST-2 phrases.

benefits more from long reviews with rich lexical signal, partially closing the gap with the transformer.

D. Efficiency Comparison

Table II and Fig. 7 present the full midterm comparison, including per-class breakdowns for the classical baseline.

The efficiency trade-off is stark. On SST-2, DistilBERT FP32 is $45\times$ slower per sample (18.5 ms vs. 0.41 ms) and occupies $\sim 966\times$ more disk space (267.9 MB vs. 0.28 MB). On IMDb, the latency ratio increases to $76\times$ (51.4 ms vs. 0.67 ms), reflecting the additional computation from longer sequences (`max_length`= 256 vs. 64).

The accuracy gains of +14.0 pp on SST-2 and +6.25 pp on IMDb are meaningful for high-accuracy applications such as medical sentiment or legal document analysis, but for low-latency pipelines the classical baseline remains strongly competitive. INT8 quantisation (Week 3) will target a 30 to 40 % reduction in DistilBERT latency, potentially narrowing the gap to $\sim 30\times$ on SST-2 and $\sim 50\times$ on IMDb.

E. N-gram Ablation

Table III reports a validation-set ablation over n-gram ranges on SST-2. Unigrams + bigrams (1,2) achieve the best validation accuracy (72.0 %) and macro-F1 (0.7190). Trigrams provide a marginal improvement over unigrams but fall short of bigrams, consistent with the expectation that SST-2’s short phrases (median 7 words) contain insufficient context to benefit from longer n-grams.

V. PLANNED WORK AND TIMELINE

INT8 Dynamic Quantisation (Week 3). Apply `torch.quantization.quantize_dynamic()` to both fine-tuned DistilBERT models, targeting all `nn.Linear`

TABLE IV
PROJECT TIMELINE

Wk	Phase	Deliverable	Status
1	Dataset & Setup	EDA, tokenisation coverage, seeds	Done
2	Baseline & FT	LR baseline, DistilBERT FT, midterm report	Done
3	Compression	INT8 quant., sequence-length ablation, size ablation	Active
4	Analysis & Report	Layer-freeze curve, error analysis, final report	Planned

layers. Expected outcome: ≈ 30 to 40 % latency reduction and $\approx 4\times$ size reduction with < 1 pp accuracy loss.

Layer-Freezing Efficiency Curve (Weeks 3 and 4). This is the project novelty. We train five DistilBERT variants by freezing 0, 1, 2, 3, or 4 bottom transformer layers and measuring validation F1, training time, and CPU inference latency for each. The resulting Pareto frontier (F1 vs. latency) on both datasets constitutes the original contribution of this project. We hypothesise that freezing the bottom 2 layers costs < 0.5 pp F1 while reducing latency by 15 to 25 %.

Sequence-Length Ablation (Week 3). Sweep `max_length` $\in \{64, 128, 256, 512\}$ on IMDb to produce a coverage-latency-accuracy Pareto curve, isolating the optimal operating point for the final report.

Training-Set Size Ablation (Week 3). Vary training subset size (500, 1K, 2K, 4K, full) to characterise sample efficiency of DistilBERT versus TF-IDF+LR. Expected: the transformer’s advantage diminishes at very low data regimes.

Cross-Dataset Evaluation (Week 4). Test the SST-2-fine-tuned model on IMDb and vice versa, quantifying domain-transfer robustness.

Error Analysis (Week 4). Inspect misclassified examples focusing on negation handling (“*not good*”), sarcasm, and domain-specific vocabulary using the analysis framework described in [4].

Full Held-Out Evaluation (Week 4). Evaluate on the official IMDb test set (25,000 samples) and SST-2 validation set (872 samples) with final fine-tuned and quantised models.

Risks and Mitigations. Training DistilBERT on IMDb with `max_length`= 256 takes ≈ 11 to 12 min per two-epoch run on the available CPU; caching checkpoints with `RUN_TRAINING=False` flags mitigates repeated re-training cost. Dynamic INT8 quantisation may produce instability on small validation subsets; if the accuracy drop exceeds 2 pp, static quantisation with a calibration set is the planned fallback. Floating-point non-determinism across hardware will be mitigated by reporting mean $\pm$ std over three seeds in the final report.

VI. TEAM CONTRIBUTIONS

Arwa Elgazar: Data and Classical Baseline. Loaded both SST-2 and IMDb datasets; performed EDA (Figs. 1

Midterm Results: DistilBERT vs TF-IDF + Logistic Regression

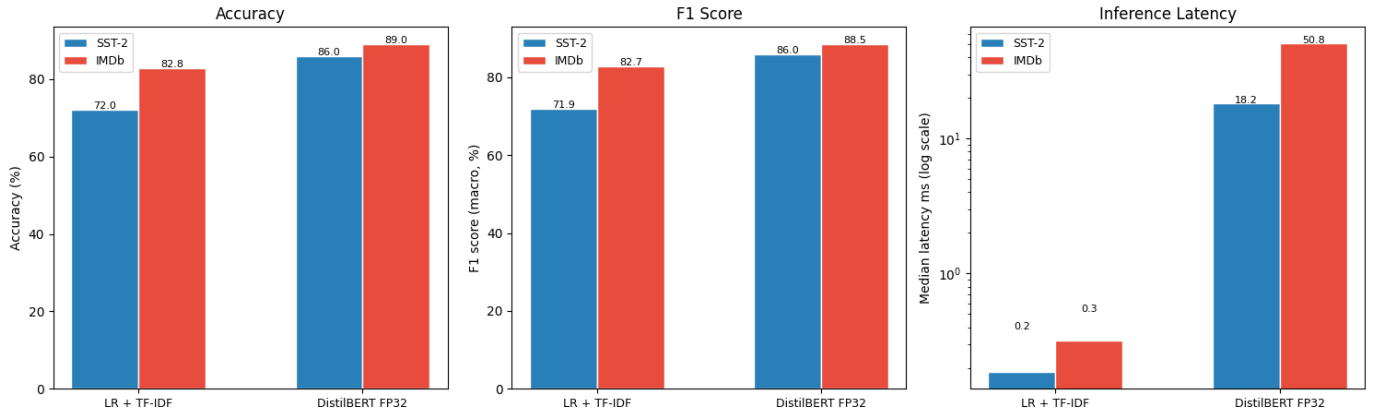


Fig. 7. Midterm comparison across all four model and dataset combinations: accuracy, macro-F1, and median per-sample inference latency (log scale). DistilBERT consistently outperforms LR+TF-IDF on both accuracy metrics at the cost of substantially higher latency.

and 2); implemented the `clean_imdb()` HTML preprocessing pipeline; conducted token-coverage analysis to select `max_length` parameters; built, trained, and benchmarked the TF-IDF+LR baseline on both datasets including latency measurement and feature-importance plots; performed n-gram range ablation (Table III). **Future:** cross-dataset evaluation, error analysis, final report Sections 3 and 4.

Eman Elsayed: DistilBERT Fine-Tuning. Implemented DistilBERT tokenisation and HuggingFace Dataset wrapping; configured the `Trainer` with appropriate hyperparameters and callbacks; ran fine-tuning on both subsets (2 epochs each); extracted and plotted training curves from `trainer.state.log_history`; resolved the `eval_strategy/evaluation_strategy` compatibility issue in Transformers 5.x. **Future:** INT8 dynamic quantisation (Week 3), layer-freezing experiment, sequence-length ablation.

Esraa Nematalla: Efficiency Benchmarks and Report. Designed and implemented the latency benchmark (100 samples, 5 warm-up, median and p95); measured model size via pickle byte count (LR) and `torch.save(state_dict())` byte count (DistilBERT); assembled the midterm comparison table and visualisation (Fig. 7); authored this report and maintained the project log. **Future:** memory profiling with `psutil` RSS, training-size ablation, Pareto-frontier analysis, final report Sections 5 through 8.

VII. USE OF GENAI TOOLS

The following describes all uses of generative AI tools in this project.

- **Concept clarification:** GenAI was used to clarify the mechanics of DistilBERT’s triple distillation loss (soft targets, cosine embedding alignment, masked LM) after reading Sanh et al. [2]. No generated text appears in this report.

- **Debugging:** After encountering a `Trainer TypeError` (evaluation_strategy/save_strategy version mismatch in Transformers 5.x), GenAI was consulted after an independent fix attempt; the correction was verified against the Transformers changelog.
- **Project scaffolding:** GenAI assisted with the structural design of the experiment pipeline (CFG dict, data-split helpers, latency measurement protocol) during initial project setup; all code was reviewed and adapted to the team’s notebook before execution.
- **Grammar:** Minor English phrasing suggestions were accepted in some report sentences. All technical content, experimental design, and analysis are original.

No experimental results, figures, or tables were produced by AI.

REFERENCES

- [1] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *Proc. NIPS Deep Learning and Representation Learning Workshop*, 2015.
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter,” *arXiv:1910.01108*, 2019. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, pp. 4171 to 4186, 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [4] A. Maas, R. Daly, P. Pham, D. Huang, A. Ng, and C. Potts, “Learning word vectors for sentiment analysis,” in *Proc. ACL*, pp. 142 to 150, 2011. [Online]. Available: <https://ai.stanford.edu/~amaas/data/sentiment/>
- [5] R. Socher et al., “Recursive deep models for semantic compositionality over a sentiment treebank,” in *Proc. EMNLP*, pp. 1631 to 1642, 2013.
- [6] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP (System Demos)*, pp. 38 to 45, 2020. [Online]. Available: <https://github.com/huggingface/transformers>